

Práctica de Traductores de Lenguajes: Memoria

Grupo 7:

Álvaro García Corral

Andrés Oreiro Santomé

Haodi Lin Sun

Índice

1. Introducción.....	4
2. Distribución del Traductor.....	4
2.1 Módulos de Traductor	4
2.2 Procesador.....	6
2.3 Generador de Código Intermedio	6
2.4 Generador de Código Objeto	7
2.5 Script de compilación	8
3. Diseño de Generador del Código Intermedio	8
3.1 Procedimiento inicial	8
3.2 Estructuras de datos	8
3.3 Métodos.....	10
3.3.1 Lectura de ficheros	10
3.3.2 Método de generación de cuartetos	10
3.3.3 Proceso de sentencias.....	11
3.3.4 Evaluar expresiones.....	12
3.3.5 Métodos auxiliares.....	12
3.4 Estructura general de programa	12
3.5 Traducción generada por sintaxis.....	14
3.6 Tabla de operadores.....	16
3.7 Clases de argumentos	16
3.8 Librerías empleadas.....	17
4.Diseño de Generador de Código Objeto.....	17
4.1 Arquitectura general del GCO	17
4.2 Lectura y parseo de los cuartetos	18
4.3 Primera pasada: recolección de metadatos	18
4.4 Resolución de operandos según su clase	19
4.5 Plantillas de traducción por tipo de cuarteto	20
4.6 Tratamiento de cadenas y subrutinas auxiliares	21
4.7 Almacenamiento de los datos: el segmento de datos.....	22
5. Diseño de Registros de Activación	22
5.1 Estructura del Registro de Activación	22

5.2 Cálculo del tamaño del RA en tiempo de compilación	23
5.3 Acceso a variables según su ámbito	23
5.4 Paso de parámetros por valor	24
5.5 Llamada a función	24
5.6 Retorno y recogida del valor devuelto	25
6.Casos de prueba.....	25
7.Conclusiones	26
8.Anexo	26

1. Introducción

2. Distribución del Traductor

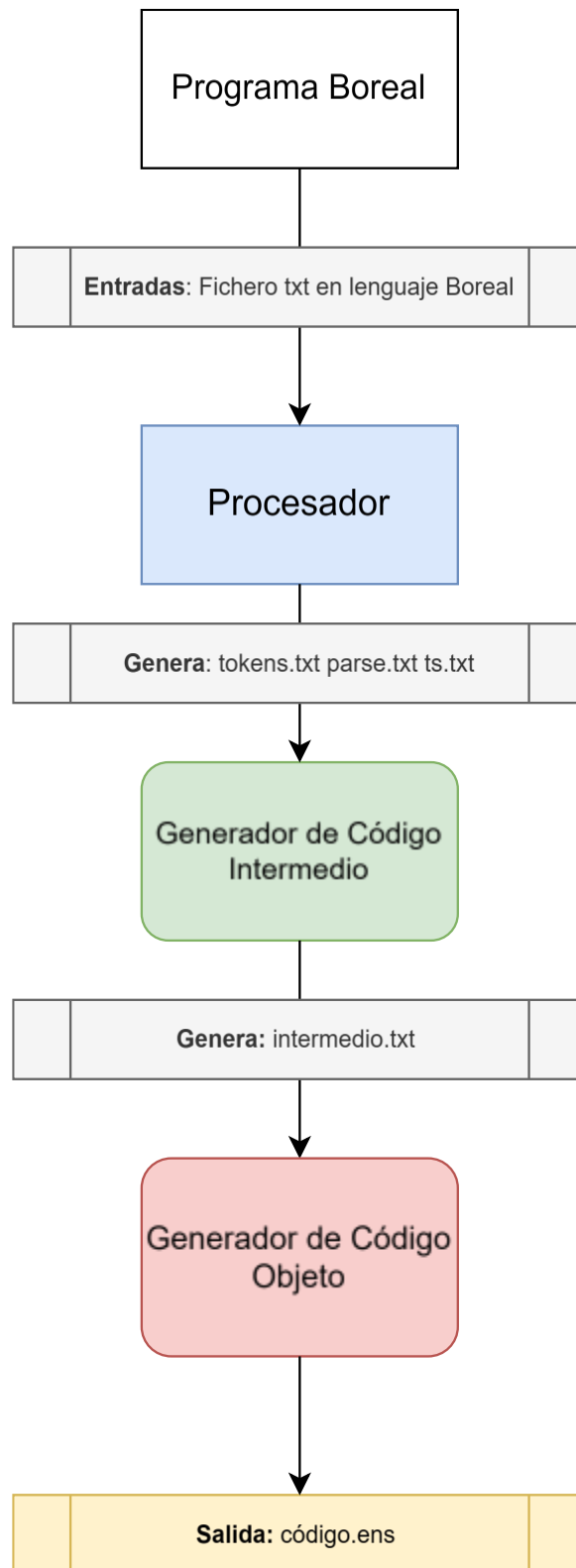
2.1 Módulos de Traductor

Nuestro traductor sigue una arquitectura dividida en tres fases independientes:

Análisis de parte de procesador, Generación de Código intermedio y Generación de Código Objeto. Cada fase se implementa de manera separada, comunicada por ficheros de texto, permitiendo el desarrollo y facilita la depuración del código.

El **Procesador**, es el módulo proporcionado por el profesor que realiza el análisis léxico, sintáctico y semántico del programa fuente en lenguaje Boreal. Se generan varios ficheros de tokens que contienen la secuencia, parse (reglas gramaticales en un orden específico), tabla de símbolos con información de tipos y desplazamientos, y un registro de errores, todos con extensión txt.

En el siguiente diagrama, mostramos el flujo completo del traductor:



2.2 Procesador

El procesador es el módulo proporcionado por los profesores de la asignatura, implementado en Java. Se encarga de realizar el análisis léxico, sintáctico y semántico.

El analizador léxico lee el fichero de entrada carácter a carácter, transformándolo en una secuencia de Tokens. Cada token representa un elemento del lenguaje: palabras reservadas (PROGRAM, VAR, BEGIN, IF), identificadores, constantes numéricas, cadenas de caracteres, operadores y signos de puntuación. El resultado del análisis se almacenará en el fichero tokens.txt, donde cada línea contendrá el token en el formato <TIPO, valor>

El analizador sintáctico toma la secuencia de tokens y verificará que se cumplen los estándares de la gramática del lenguaje Boreal. Utilizamos entonces un analizador descendente con tablas LL(1), cargando la tabla desde el fichero tabla_acc.htm en la carpeta resources. Mientras que se aplican las reglas gramaticales, se registrará en el parse.txt la secuencia de números de regla que se ha seguido. La información es esencial para que el Generador de Código Intermedio pueda reconstruir la estructura del programa.

Por último, el analizador semántico ejecutará con el sintáctico las acciones semánticas asociadas a cada regla gramatical. Gestiona la tabla de símbolos (con identificador, tipo, desplazamiento, ámbito (local o global), y número con parámetros, el tipo de retorno y etiqueta para los subprogramas). Se comprueba si es compatible o no los tipos en asignaciones, expresiones, paso de parámetros... Toda la información se vuelca en el fichero ts.txt. Los errores detectados se registrarán en errores.txt

Comandos para ejecutar el procesador:

- `javac -cp "lib/*" -d bin src/procesador/*.java src/tslib/*.java`
- `java -cp "bin;lib/*" procesador.Procesador ejemplos/prueba1.txt`

Nota: Los ficheros que se utilicen para probar nuestro traductor se localizarán en la carpeta de AnalizadorTablas/ejemplos

2.3 Generador de Código Intermedio

El GCI es el módulo que toma como entrada los ficheros generados por el Procesador y produce un fichero de código intermedio en formato de cuartetos. El proceso se basa en recorrer secuencialmente los tokens, detectar construcciones como asignaciones, condicionales, E/S, y emitirá los cuartetos correspondientes

Ejemplo de formato de cuarteto: (OP, {CLASE, valor}, {CLASE, valor}, {CLASE, valorResultado})

Ejecución en CMD con:

```
javac -cp "lib/*" -d bin src/procesador/*.java src/tslib/*.java (Para compilar en terminal)
```

```
java -cp "bin;lib/" procesador.Procesador ejemplos/prueba1.txt
```

2.4 Generador de Código Objeto

Está desarrollado en Python. Traduce los cuartetos del código intermedio a código ensamblador ENS2025.

El fichero entrada es el fichero salida del código intermedio: intermedio.txt que contiene los cuartetos extraídos de los ficheros del procesador.

El generador lee cada línea de fichero de cuartetos, y extrae el operador y argumentos con sus clases y valores. Una vez identificadas las funciones, calcula los tamaños del Registro de Activación, detecta variables globales y registra las constantes de cadena.

Se aplica finalmente una plantilla de traducción a cada operador, recorriendo previamente los cuartetos para generar las instrucciones ENS2025 correspondientes

Ejecución en CMD con:

```
python gco.py intermedio.txt codigo.ens
```

2.5 Script de compilación

Hemos creado un script para facilitar la ejecución secuencial de los módulos:
compilador.bat →

```
echo Compilador procesador
```

```
javac -cp "lib/*" -d bin src/procesador/*.java src/tslib/*.java
```

```
java -cp "bin;lib/*" procesador.Procesador ejemplos/prueba1.txt
```

```
echo Compilador codigo intermedio
```

```
java -cp "bin;lib/*" procesador.GeneradorCodigoIntermedio tokens.txt parse.txt  
ts.txt intermedio.txt
```

```
echo Compilador codigo objeto
```

```
python gco.py intermedio.txt codigo.ens
```

3. Diseño de Generador del Código Intermedio

3.1 Procedimiento inicial

EL GCI funciona como un módulo independiente que procesa la salida del Procesador. El procedimiento consiste en recorrer secuencialmente la lista de tokens generada por el analizador léxico, detectando patrones sintácticos y generando cuartetos correspondientes. Para la evaluación de expresiones, se utiliza un algoritmo de pila de precedencia de operadores. (Pila de operandos y operadores).

3.2 Estructuras de datos

Para almacenar la información necesaria, el GCI define dos clases auxiliares: TokenInfo y VarInfo.

TokenInfo: Representa un token individual, extractor de tokens.txt. Almacena tres campos:

- Tipo: cadena que identifica la categoría del token. Puede ser:
ID, ENTERO, CADENA, ASIGNACION, MAS, MENOS, AND, IGUAL, PYC, IF, THEN,

REPEAT, UNTIL, WRITE, READ, RETURN, BEGIN, END, PARENT_ABRIR, PARENT_CERRAR, TRUE, FALSE...

- **Valor:** Contiene el valor léxico del token, dependiendo de qué tipo sea el token, el valor varía. Ejemplo: Para un ID, el valor será la posición del token en la tabla de símbolos, para un entero será un valor numérico, o para una cadena, será un String con comillas.

VarInfo: Representa una entrada de la Tabla de Símbolos, construida a partir de ts.txt. Nos permite conocer el tipo y dirección de cada variable. Contiene 7 campos:

- **Nombre:** cadena con el lexema del identificador. Se utiliza como clave en el mapa tablaSímbolos
- **Tipo:** Indica el tipo de dato del identificador. Los valores puede ser entero, cadena, procedimiento o lógico.
- **Desplazamiento:** Indica la dirección relativa de una variable dentro de su ámbito.
- **esGlobal:** Indica si la variable es global o no mediante un booleano.
- **esParametro:** Indica si la variable es un parámetro de una función mediante booleano.
- **Etiqueta:** Almacena la etiqueta asignada por el procesador a cada función o procedimiento.
- **posición:** guarda el orden de declaración de la variable o subprograma dentro de la tabla (global o local)

Pilas para evaluación de expresiones: Dentro del método evaluarExpresion(), Tenemos dos tipos:

- **Pila de operandos:** Almacena de los operandos ya procesados. Cada elemento es de tipo {CLASE, valor}. Cuando se reduce un operador, se extraen dos operandos de esta pila.
- **Pila de operadores:** Almacena los operadores pendientes de reducción. Contiene símbolos MAS, MENOS, AND, IGUAL. La reducción se realiza cuando un operador, tiene precedencia menor o igual que el de la cima de la pila. . De esa manera, el orden de evaluación satisface las reglas del lenguaje Boreal

Ejemplo: Para la expresión a + 5.

Estado inicial de pila:

Pila operandos: [{VAR_GLOBAL, 0}]

Pila operadores: []

Cuando mira el entero:

Pila operandos: [{VAR_GLOBAL, 0}, {CTE_ENT, 5}]

Pila operadores: [MAS]

Si encuentra el token PYC, fin de la expresión

Vacía los elementos de la lista de operandos y de operadores, y crea VAR_TEMP, que almacena el resultado de la expresión

Estado final:

Pila operandos: [{VAR_TEMP,0}]

Pila operadores: []

Listas y mapas empleados:

- ArrayList<TokenInfo> tokens: Contiene todos los tokens leídos del fichero de entrada, en el mismo orden en el que aparecen. El índice posToken recorre la lista secuencialmente
- ArrayList<Integer> reglas: Contiene la secuencia de números de regla de parse.txt.
- HashMap<String, VarInfo> tablaSimbolos: Asocia el nombre de cada variable en mayúsculas con la información correspondiente
- ArrayList<String> cuartetos: Acumula los cuartetos generados en el orden en el que se emiten. Se vuelca en el fichero de salida al final.

3.3 Métodos

3.3.1 Lectura de ficheros

Tendremos tres métodos de fines independientes:

- leerTokens: lee el fichero tokens.txt generado por el analizador léxico. Cada línea tiene el formato <TIPO, valor>. El método extrae el tipo y el valor, y para los identificadores guarda también el número de ID (positivo para variables globales, negativo para locales).
- leerParse: lee parse.txt, que contiene la secuencia de reglas aplicadas por el analizador sintáctico.
- leerTS(fich): reconstruye la Tabla de Símbolos a partir de ts.txt. Por cada entrada, se crea un objeto VarInfo que almacena el nombre, tipo, desplazamiento y ámbito (global o local). Esta información es consultada constantemente durante la generación de código para saber la clase de cada variable, y si es de tipo cadena.

3.3.2 Método de generación de cuartetos

generarCodigo(): emite la etiqueta main y recorre la lista de tokens secuencialmente.

- Si encuentra un ID con :=, llamo a procesarAsignacion()
- Si encuentro IF, REPEAT, WRITE, READ, RETURN, se llama al método correspondiente a cada una
- Si encuentro un ID con (, llamamos a un subprograma.
- En otro caso, se recorrerá al siguiente token.
- Se introduce HALT al final del programa.

3.3.3 Proceso de sentencias

- **procesarAsignacion():** traduce “id := expresion ;”. Primero guarda el token del identificador destino (el lado izquierdo). Luego llama a evaluarExpresion() para procesar el lado derecho. Finalmente, busca el desplazamiento del destino en la TS y emite ASIG (para enteros y lógicos) o ASIG_CAD (para cadenas). La decisión del operador depende del tipo del origen, no del destino, porque en Boreal no hay conversión implícita de tipos.
- **procesarIf():** Traduce la condicional simple IF algo THEN respuesta. Primero evalúa la condición, y se crean dos etiquetas, para el THEN y para el final. Se emite un salto condicional. Si la condición es cierta, se salta a etiqueta THEN, y si no, se emite un GOTO incondicional al final para saltarlo.
- **procesarRepeat():** El método que se nos ha asignado a nuestro grupo, implementa la sentencia REPEAT-UNTIL. Se emite una etiqueta al inicio, se procesa el cuerpo hasta tener UNTIL, se evalúa la condición, y se sale del bucle hasta que la condición sea verdadera.
- **procesarRead():** similar a procesarWrite pero para lectura. Por cada variable en la lista, emite INPUT_ENT o INPUT_CAD según su tipo en la TS. Solo acepta identificadores, no expresiones arbitrarias.
- **procesarWrite():** Para cada expresión en la lista de argumentos, se emitirá PRINT_ENT o PRINT_CAD según el tipo entero o cadena. Los paréntesis y comas se consumen como delimitadores y no generan código.
- **procesarReturn():** si hay una expresión después de RETURN, la evalúa y emite RETURN_ENT o RETURN_CAD. Si no hay expresión, emite RETURN (para procedimientos o el programa principal).
- **procesarLlamada():** traduce llamadas a subprogramas. Primero evalúa cada parámetro y emite PARAM o PARAM_CAD. Luego busca en la TS la etiqueta del subprograma y su tipo de retorno. Si no devuelve nada, emite CALL. Si devuelve cadena, CALL_FUN_CAD. En otro caso, CALL_FUN.

3.3.4 Evaluar expresiones

- **evaluarExpresion():** implementa el algoritmo para la pila de precedencia de operadores. Como bien se mencionó anteriormente, utiliza la pila de operandos y operadores. Recorre los

3.3.5 Métodos auxiliares.

- **buscarVariablePorToken(token):** busca por orden de declaración en una lista ordenada de variables locales para obtener los desplazamientos. El número de ID del token (positivo = global, negativo = local) se compara con los desplazamientos almacenados. Retorna el lugar en formato {VAR_GLOBAL, desp} o {VAR_LOCAL, desp}.
- **esCadena(lugar):** determina si un operando es de tipo cadena. Consulta la TS por el desplazamiento o comprueba si el lugar contiene CTE_CAD. Esta decisión es crítica para elegir entre SUMA/CONCAT, ASIG/ASIG_CAD, PRINT_ENT/PRINT_CAD, etc.
- **emitirCuarteto(op, arg1, arg2, res):** formatea un cuarteto en el formato oficial (OP, ARG1, ARG2, RES) y lo añade a la lista. Los campos vacíos se rellenan con -.
- **nuevaTemporal() y nuevaEtiqueta():** generan nombres únicos para temporales ({VAR_TEMP, 0}, {VAR_TEMP, 1}, ...) y etiquetas ({ET, "L0"}, {ET, "L1"}, ...) incrementando contadores internos.
- **guardarFichero(fich):** escribe todos los cuartetos generados al fichero de salida, uno por línea, precedidos por un comentario // Código intermedio generado.

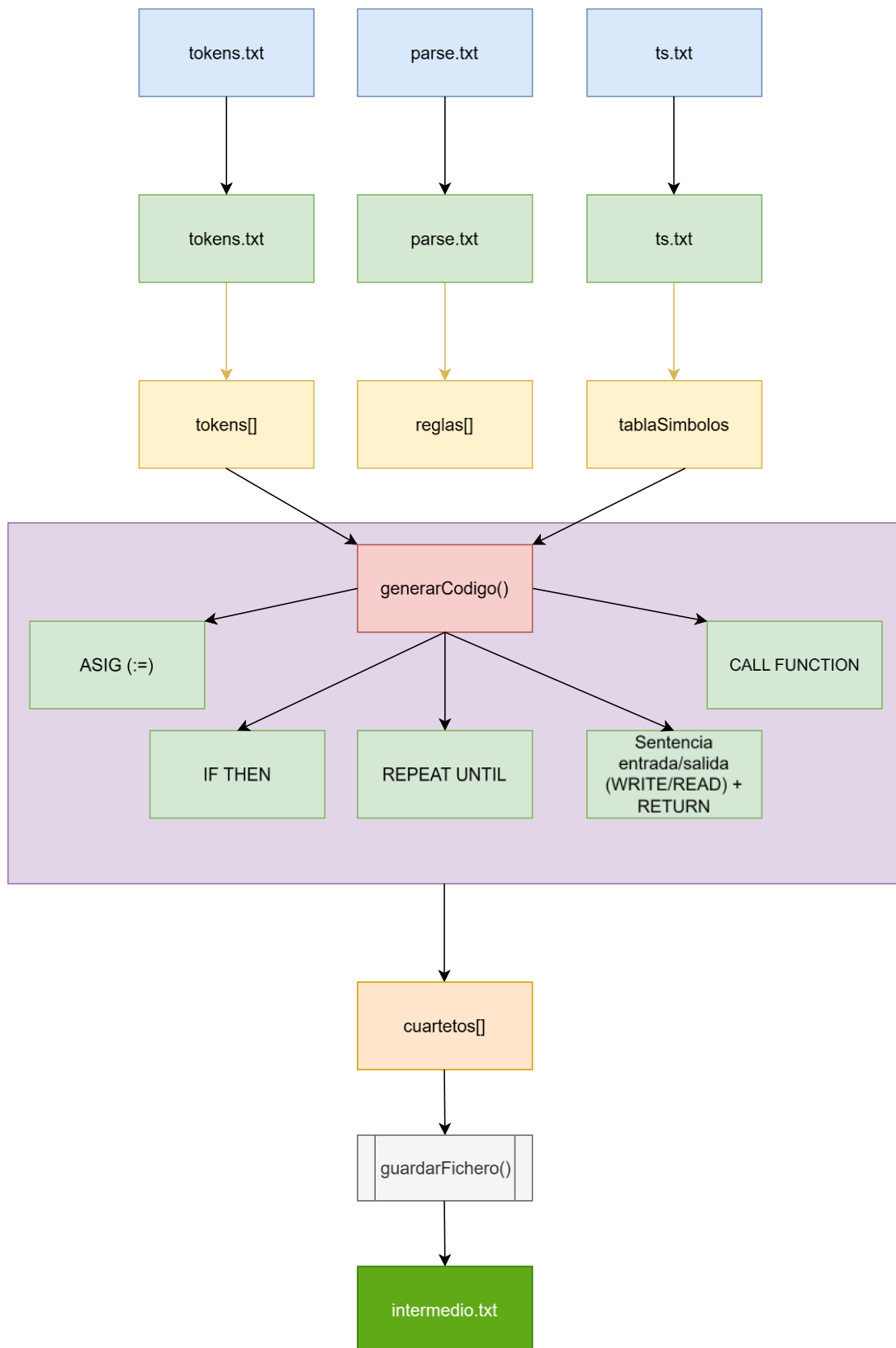
3.4 Estructura general de programa

El GCI funciona con tres fases diferenciadas:

1. Lectura: Los ficheros se leen en paralelo con los métodos leerTokens(), leerParse() y leerTS()
2. Generación: El método generarCodigo() recorre la lista de tokens, y según el tipo de token encontrado, se ejecuta el método correspondiente que consulta la tabla de Símbolos para resolver tipos y desplazamientos, y utilizan evaluarExpresion() para procesar las expresiones. El resultado se almacenará en la lista de cuartetos[]

3. El método guardarFichero() coge la lista de cuartetos[] y lo vuelca en el fichero intermedio.txt

En el siguiente diagrama, podemos apreciar el procedimiento:



3.5 Traducción generada por sintaxis

Cada construcción de lenguaje se traduce según una regla semántica fija que indica qué cuartetos emitir.

Asignación:

Regla sintáctica	Regla semántica
$S \rightarrow id := E;$	Emitir(ASIG, E.lugar, -, id.lugar)
$S \rightarrow id := E; \text{ (Para cadenas)}$	Emitir(ASIG_CAD, E.lugar, -, id.lugar)

Expresiones:

Regla sintáctica	Regla semántica
$E \rightarrow E1 + E2 \text{ (entero)}$	E.lugar = nuevaTemp(); emitir(SUMA, E1.lugar, E2.lugar, E.lugar)
$E \rightarrow E1 + E2 \text{ (cadena)}$	E.lugar = nuevaTemp(); emitir(CONCAT, E1.lugar, E2.lugar, E.lugar)
$E \rightarrow E1 - E2$	E.lugar = nuevaTemp(); emitir(RESTA, E1.lugar, E2.lugar, E.lugar)
$E \rightarrow E1 \text{ AND } E2$	E.lugar = nuevaTemp(); emitir(AND, E1.lugar, E2.lugar, E.lugar)
$E \rightarrow E1 = E2$	E.lugar = nuevaTemp(); emitir(GOTO_IG, E1.lugar, E2.lugar, E.lugar)
$E \rightarrow \text{entero}$	E.lugar = {CTE_ENT, valor}
$E \rightarrow \text{cadena}$	E.lugar = {CTE_CAD, "texto"}
$E \rightarrow \text{true}$	E.lugar = {CTE_ENT, 1}
$E \rightarrow \text{false}$	E.lugar = {CTE_ENT, 0}
$E \rightarrow id$	E.lugar = buscarTS(id)

Condicional:

Regla sintáctica	Regla semántica
S → IF E THEN S1	etiq = nuevaEtiqu(); emitir(GOTO_IG, E.lugar, 0, etiq); S1.cod; emitir(ETIQ, etiq)

REPEAT-UNTIL:

Regla sintáctica	Regla semántica
S → REPEAT S1 UNTIL E	etiq = nuevaEtiqu(); emitir(ETIQ, etiq); S1.cod; emitir(GOTO_IG, E.lugar, 0, etiq)

Entrada/Salida:

Regla sintáctica	Regla semántica
S → write(E)	Emitir(PRINT_ENT o PRINT_CAD, -, -, E.lugar)
S → read(V)	emitir(INPUT_ENT o INPUT_CAD, -, -, V.lugar)

Retorno:

Regla sintáctica	Regla semántica
S → RETURN ;	emitir(RETURN, -, -, -)
S → RETURN E ;	emitir(RETURN_ENT o RETURN_CAD, -, -, E.lugar)

Llamada a subprograma:

Regla sintáctica	Regla semántica
S → id(L)	emitir(PARAM, ...); emitir(CALL, etiq, -, -)
E → id(L)	E.lugar = nuevaTemp(); emitir(PARAM, ...); emitir(CALL_FUN, etiq, -, E.lugar)

3.6 Tabla de operadores

Operador	Significado
SUMA	Suma entera
RESTA	Resta entera
AND	AND lógico
CONCAT	Concatenación de cadenas
ASIG	Asignación entero/lógico
ASIG_CAD	Asignación cadena
GOTO	Salto incondicional
GOTO_IG	Salto si igual
ETIQ	Etiqueta
PARAM	Paso de parámetro
PARAM_CAD	Paso de parámetro cadena
CALL	Llamada a procedimiento
CALL_FUN	Llamada a función
CALL_FUN_CAD	Llamada a función (retorna cadena)
RETURN	Retorno sin valor
RETURN_ENT	Retorno de entero
RETURN_CAD	Retorno de cadena
PRINT_ENT	Imprimir entero
PRINT_CAD	Imprimir cadena
INPUT_ENT	Leer entero
INPUT_CAD	Leer cadena
HALT	Fin de programa

3.7 Clases de argumentos

Clase	Significado	Ejemplo
VAR_GLOBAL	Variable global, desplazamiento desde .IY	{VAR_GLOBAL, 5}
VAR_LOCAL	Variable local, desplazamiento desde .SP	{VAR_LOCAL, 3}
VAR_TEMP	Variable temporal	{VAR_TEMP, 0}
CTE_ENT	Constante entera	{CTE_ENT, 42}
CTE_CAD	Constante cadena	{CTE_CAD, "hola"}
ET	Etiqueta	{ET, "main"}
PAR	Parámetro por valor	{PAR, 1}

3.8 Librerías empleadas

Java.io.* : Para la lectura de ficheros tokens.txt, parse.txt y ts.txt, utilizamos clases más didácticas: `BufferedReader` para la lectura y `PrintWriter` para la escritura en `intermedio.txt`

Java.util.*: Para las estructuras de datos `ArrayList`, `HashMap` y `Stacks` (Pilas). En este caso no utilizamos `Arrays` para las pilas siendo mejor para representar el orden de evaluación LIFO

4. Diseño de Generador de Código Objeto

El Generador de Código Objeto constituye la última fase del traductor. Recibe como entrada el fichero de cuartetos generado por el Generador de Código Intermedio y produce como salida un programa equivalente en lenguaje ensamblador ENS2025, que posteriormente se ensambla para obtener el código máquina ejecutable sobre la máquina virtual de 16 bits. El módulo está implementado en Python y se invoca mediante la orden «`python gco.py intermedio.txt codigo.ens`».

La estrategia que hemos seguido es la del GCO a través de la traducción mediante plantillas. A cada código de operación del cuarteto le corresponde un patrón fijo de instrucciones ensamblador. Ahora bien, como los operandos de un cuarteto pueden ser de muy distinta naturaleza como variable global, variable local, constante entera, etc. La dirección de cada operando se calcula de forma diferente en cada caso; por ello la plantilla se particulariza según la clase de cada argumento.

4.1 Arquitectura general del GCO

El GCO funciona en tres fases diferenciadas. La separación en fases es necesaria porque la traducción de un cuarteto requiere conocer información que solo está disponible tras haber recorrido el fichero completo. Por ejemplo, el tamaño del registro de activación de una función o el conjunto de constantes de cadena.

- Fase 1 Parseo. Se lee el fichero línea a línea, se descartan los comentarios (//) y las líneas vacías, y cada cuarteto textual se transforma en una estructura interna (un objeto “Quadruplet” con operador, arg1, arg2 y resultado). Cada argumento se representa con su clase y su valor o valores correspondientes.
- Fase 2 Primera pasada, la recolección de metadatos. Se recorre la lista completa de cuartetos para extraer información global: qué etiquetas son funciones, el tamaño del registro de activación de cada una, el desplazamiento máximo de las variables globales, qué funciones manejan cadenas y el registro de todas las constantes de cadena.
- Fase 3 Traducción. Se recorre de nuevo la lista de cuartetos y, para cada uno, se aplica la plantilla correspondiente a su operador, emitiendo las instrucciones ENS2025. Al terminar se genera el segmento de datos.

El programa objeto comienza siempre con una cabecera fija que inicializa los registros índice y el puntero de pila y salta al programa principal:

```
ORG 0
    MOVE #inicio_estaticas, .IY    ; IY -> inicio de la zona de variables globales
    MOVE #inicio_pila, .SP         ; SP -> cima de la pila mediante memoria
                                   dinamica
    BR /main                       ; salta al programa principal
```

El registro índice IY marca el comienzo de la zona de datos estáticos, de modo que la dirección de cada variable global es relativa a IY. El registro SP, puntero pila, marca la cima de la pila de registros de activación. Tras la cabecera se traducen todos los cuartetos y, al final, se emite el segmento de datos y la pseudoinstrucción END.

4.2 Lectura y parseo de los cuartetos

El formato de cada cuarteto es «(Operador, Arg1, Arg2, Result)», donde cada argumento es una tupla «{Clase, valor}». El parser respeta las comas que separan los cuatro campos sin confundirlas con las comas internas de las tuplas, para lo cual lleva la cuenta de las llaves abiertas y de si se está dentro de una cadena entre comillas. Los comentarios «//» se eliminan respetando las comillas para no truncar una cadena que contenga «//». Cada argumento se descompone en su clase y sus valores, que pueden ser enteros, cadenas entre comillas o, en el caso de las variables no locales, un par profundidad-desplazamiento.

4.3 Primera pasada: recolección de metadatos

Antes de traducir, la primera pasada calcula toda la información que depende del programa completo:

- Identificación de funciones: se recopilan los nombres que aparecen como destino de los cuartetos CALL, CALL_FUN y CALL_FUN_CAD, y se añade «main». Cada cuarteto ETIQ cuyo nombre coincide con una función marca el inicio de su ámbito.
- Cálculo de desplazamientos máximos: por cada variable se actualiza el desplazamiento máximo de su ámbito (la zona global, o la función actual para variables locales, temporales y parámetros). Este valor permite dimensionar tanto la zona estática como cada registro de activación.
- Detección de cadenas: si una función contiene operaciones de cadena (CONCAT, ASIG_CAD, INPUT_CAD, PARAM_CAD, PRINT_CAD, CALL_FUN_CAD o RETURN_CAD) se marca que «maneja cadenas», lo que reserva un buffer adicional en su registro de activación.
- Registro de constantes de cadena: a cada constante de cadena (CTE_CAD) se le asocia una etiqueta única (_str_0, _str_1, ...) que después se volcará en el segmento de datos.

Con estos datos se calcula el tamaño del registro de activación de cada función y el tamaño de la zona global, según se detalla en la sección 5.

4.4 Resolución de operandos según su clase

El corazón del GCO es la rutina que, dada la clase de un argumento, genera la forma de direccionamiento ENS2025 adecuada para acceder a su valor. La siguiente tabla resume esa correspondencia (el desplazamiento d es el que proporciona la Tabla de Símbolos):

Clase de argumento	Direccionamiento ENS2025	Explicación
VAR_GLOBAL d	#d[.IY]	Indexado a IY: relativo al inicio de la zona estática.
VAR_LOCAL / VAR_TEMP / PAR d	#{d+2}[.SP]	Indexado a SP; se suman 2 por el EM y el PA del RA.
CTE_ENT v	#v	Inmediato: el valor va en la propia instrucción.
CTE_CAD	/_str_n	Directo a memoria: dirección de la cadena en la zona DATA.
ET nombre	/nombre	Directo a memoria: dirección de la etiqueta.
PAR_REF d	MOVE #{d+2}[.SP], .R7 [.R7]	Indirección: en el RA está la dirección del dato.
VAR_NOLOCAL p,d	MOVE #1[.SP], .IX MOVE #1[.IX], .IX (x p-1) #{d+2}[.IX]	Recorrido del enlace estático (PA) p niveles.

El motivo de sumar 2 al desplazamiento de las variables locales es que, en la base de cada registro de activación, las dos primeras palabras están ocupadas por el Estado de la Máquina (dirección de retorno) y el Puntero de Acceso (enlace estático); las variables

empiezan justo después. Como el diseño del registro de activación es conocido en tiempo de compilación, este desplazamiento fijo se suma al generar cada instrucción.

4.5 Plantillas de traducción por tipo de cuarteto

Una vez resuelta la dirección de los operandos, la traducción de cada cuarteto es directa. La siguiente tabla recoge las plantillas de los cuartetos más representativos. En las operaciones aritméticas y lógicas, ENS2025 deja el resultado en el acumulador A, por lo que siempre se añade un MOVE .A para copiarlo al destino.

Cuarteto	Plantilla ENS2025	Significado
(SUMA, y, z, x)	ADD y, z MOVE .A, x	$x = y + z$
(RESTA, y, z, x)	SUB y, z MOVE .A, x	$x = y - z$
(MUL / DIV / MOD ...)	MUL y, z MOVE .A, x	$x = y * z$ (análogo DIV, MOD)
(AND / OR, y, z, x)	AND y, z MOVE .A, x	$x = y \text{ AND } z$
(MENOS, y, , x)	MOVE y, x NEG x	$x = -y$ (menos unario)
(ASIG, y, , x)	MOVE y, x	$x = y$
(GOTO, , , et)	BR /et	salto incondicional
(GOTO_IG, x, y, et)	CMP x, y BZ /et	if $x == y$ goto et
(GOTO_DIST, x, y, et)	CMP x, y BNZ /et	if $x != y$ goto et
(GOTO_MEN, x, y, et)	CMP x, y BN /et	if $x < y$ goto et
(GOTO_MEN_IG, x, y, et)	CMP x, y BNP /et	if $x \leq y$ goto et
(GOTO_MAY_IG, x, y, et)	CMP x, y BP /et	if $x \geq y$ goto et
(PRINT_ENT, , , x)	WRINT x	escribe el entero x
(PRINT_CAD, , , x)	WRSTR /x	escribe la cadena x
(INPUT_ENT, , , x)	ININT x	lee un entero en x
(INPUT_CAD, , , x)	INSTR x	lee una cadena en x
(HALT, , ,)	HALT	detiene la ejecución

El caso del salto «mayor estricto» “GOTO_MAY” como ENS2025 no dispone de una bifurcación directa para «estrictamente positivo», se genera una comparación seguida de un salto «no positivo» a una etiqueta de escape y un salto incondicional al destino, de modo que solo se salta cuando el resultado es estrictamente positivo. El operador lógico NOT se traduce comparando el operando con cero y normalizando el resultado a 0 o 1.

Cada cuarteto “ETIQ” se traduce simplemente emitiendo la etiqueta «nombre:». Si la etiqueta corresponde a una función, se actualiza el ámbito actual y se reinicia el contador de parámetros. Las etiquetas internas que el GCO necesita generar para bucles, condicionales y subrutinas de cadena, se crean con un contador propio para garantizar que sean únicas, lo que es imprescindible cuando hay bucles repeat-until anidados.

4.6 Tratamiento de cadenas y subrutinas auxiliares

La concatenación de cadenas y la asignación de cadenas son componentes propios de las especificaciones de nuestro grupo. A diferencia de las operaciones con enteros, una cadena no cabe en un registro, es una secuencia de caracteres en memoria terminada en el carácter nulo con valor 0. Por ello, su tratamiento requiere subrutinas de copia carácter a carácter generadas en línea.

Para la asignación de cadena “ASIG_CAD” se obtiene la dirección de la cadena origen y de la cadena destino y se copian los caracteres hasta encontrar el terminador, cerrando con un 0 en el destino:

```
    ; .R0 = direccion origen, .R2 = direccion destino
_scpy:
    CMP [.R0], #0
    BZ /_scpy_e
    MOVE [.R0], .R3
    MOVE .R3, [.R2]    ; copia un caracter
    INC .R0
    INC .R2
    BR /_scpy
_scpy_e:
    MOVE #0, [.R2]    ; terminador nulo en el destino
```

La concatenación “CONCAT”, que traduce el operador «+» entre cadenas, es una extensión natural: copia primero todos los caracteres de la primera cadena al destino y, a continuación, los de la segunda, terminando con el carácter nulo. Tanto en la asignación como en la concatenación, las direcciones de origen y destino se obtienen con una rutina específica que, según la clase del operando, calcula la dirección efectiva (sumando el desplazamiento a IY o a SP, resolviendo la indirección de un parámetro por referencia o tomando la etiqueta de una constante de cadena).

Un ejemplo del operador de concatenación «+» es la sentencia «saludo := saludo + "!"»; que genera el cuarteto (CONCAT, saludo, "!", t), seguido de (ASIG_CAD, t, , saludo). El GCO traduce el CONCAT volcando primero los caracteres de «saludo» y después los de la constante «!» sobre una cadena temporal de destino, y a continuación copia el resultado de vuelta a la variable «saludo» con la subrutina de asignación de cadena descrita arriba.

4.7 Almacenamiento de los datos: el segmento de datos

El GCO decide dónde reside cada tipo de dato en la memoria de la máquina virtual:

- Las constantes de cadena se almacenan al final del programa, en la zona DATA, cada una con su etiqueta (`_str_n: DATA "texto"`). Las referencias a una constante de cadena se traducen como la dirección de su etiqueta.
- Las variables globales residen en la zona de datos estáticos, reservada con «`inicio_estaticas: RES tamaño`» y apuntada por el registro `IY`. Su dirección es relativa a `IY`.
- Variables locales, temporales y parámetros residen en el registro de activación de la función en la pila, y su dirección es relativa a `SP` (véase la sección 5).
- Pila de registros de activación comienza en la etiqueta «`inicio_pila`», situada justo detrás de la zona estática.

Un ejemplo de segmento de datos generado al final del fichero objeto sería:

```
; Segmento de datos
_str_0: DATA "Hola"
_str_1: DATA "Mundo"
inicio_estaticas: RES 12      ; reserva la zona de variables globales
inicio_pila:                ; comienzo de la pila
END
```

De este modo, `IY` apunta a «`inicio_estaticas`» y `SP` a «`inicio_pila`», ambos valores conocidos tras ensamblar, lo que garantiza que las variables globales y la pila ocupen posiciones de memoria contiguas y no solapadas con la zona de código.

5. Diseño de Registros de Activación

Para dar soporte a las funciones y subprogramas, el GCO organiza la memoria dinámica como una pila de Registros de Activación. Cada vez que se invoca a una función se reserva un RA en la cima de la pila, que contiene toda la información propia de esa invocación. El registro `SP` apunta a la base del RA activo y el acceso a las variables locales se realiza siempre de forma relativa a `SP`. El diseño del RA es conocido en tiempo de compilación, por lo que todos los desplazamientos se calculan al generar el código.

5.1 Estructura del Registro de Activación

En la solución del grupo 7 no se utiliza Puntero de Control; el RA contiene únicamente el Estado de la Máquina y el Puntero de Acceso como cabecera fija de 2 palabras, seguidos de la zona de parámetros y variables. La estructura de un RA, desde la posición a la que apunta `SP`, es la siguiente:

Desplazamiento (respecto a SP)	Campo	Contenido
0	EM (Estado de la Máquina)	Dirección de retorno al llamador.
1	PA (Puntero de Acceso)	Enlace estático: apunta al RA del bloque padre.
2, 3, ...	Parámetros y variables	Parámetros por valor, variables locales y temporales, en el orden de la Tabla de Símbolos.
...	Buffer de cadenas	64 palabras adicionales, solo si la función maneja cadenas.

La cabecera fija (EM + PA) ocupa, por tanto, 2 palabras. Es por eso que la dirección de una variable local de desplazamiento d es « $\#(d+2)[.SP]$ » donde las dos primeras posiciones están reservadas y las variables comienzan en el desplazamiento 2. Las variables temporales y los parámetros por valor reciben exactamente el mismo tratamiento que las variables locales, ya que también residen en el RA activo.

5.2 Cálculo del tamaño del RA en tiempo de compilación

El tamaño de cada RA se calcula en la primera pasada a partir del desplazamiento máximo de las variables de la función. La fórmula empleada es:

```
Tam_RA = (desplazamiento_maximo + 1)      ; espacio de parametros y variables
          + 64 (solo si maneja cadenas)    ; buffer de cadenas
          + 2                             ; Estado de Maquina + Puntero de
                                         Acceso
```

Por otro lado, el tamaño de la zona de variables globales es el desplazamiento máximo global más uno, ampliado con un buffer de cadenas si existen cadenas globales.

5.3 Acceso a variables según su ámbito

El ámbito de una variable determina cómo se calcula su dirección, tal y como se resumió en la sección 4.4:

- Variables globales están en la zona estática; su dirección es « $\#d[.IY]$ ». Por ejemplo, la asignación de la constante 0 a una global de desplazamiento 5 se traduce como « $\text{MOVE } \#0, \#5[.IY]$ ».
- Variables locales, temporales y parámetros por valor están en el RA activo; su dirección es « $\#(d+2)[.SP]$ ».
- Parámetros por referencia en el RA no está el dato sino su dirección, por lo que es necesaria una indirección, primero se carga la dirección en un registro y luego se accede al dato a través de él.

- Variables no locales residen en un RA distinto del activo; se accede a ellas recorriendo el enlace estático (los Punteros de Acceso) tantos niveles como indique la diferencia de profundidades, usando el registro índice IX.

5.4 Paso de parámetros por valor

El paso de parámetros por valor es uno de los componentes de nuestro grupo. El cuarteto PARAM coloca el valor del argumento en la zona de parámetros del RA del llamado, que todavía no es el RA activo (SP aún apunta al RA del llamador). Como ENS2025 no puede calcular esa posición en una sola instrucción, se hace en dos pasos, apoyándose en el acumulador:

```
ADD #(Tam_RA_llamador + 2 + pos_parametro), .SP
MOVE valor, [.A]      ; coloca el valor en la zona de parametros del nuevo
                      RA
```

El desplazamiento del parámetro dentro del nuevo RA no se puede obtener de la Tabla de Símbolos, de modo que se lleva en un contador interno que se incrementa con cada parámetro pasado. Para los parámetros de tipo cadena “PARAM_CAD” no se copia un único valor, sino toda la cadena carácter a carácter, y el contador avanza el tamaño del buffer de cadenas de 64 palabras.

5.5 Llamada a función

La traducción de una llamada (CALL para procedimientos, CALL_FUN / CALL_FUN_CAD para funciones que devuelven valor) realiza los siguientes pasos, donde Tam_RA_llamador es el tamaño del RA de la función que efectúa la llamada:

1. Guardar la dirección de retorno en el campo EM del RA del llamado: «MOVE #ret, #Tam_RA_llamador[.SP]», siendo «ret» una etiqueta generada para la instrucción siguiente a la llamada.
2. Establecer el Puntero de Acceso (enlace estático). Se recorren «prof_llamador – prof_llamado + 1» punteros de acceso desde el RA actual y la dirección resultante se guarda en el campo PA del RA del llamado («#(Tam_RA_llamador+1)[.SP]»). El código generado depende de la relación de anidamiento entre la función que llama y la llamada.
3. Activar el nuevo RA incrementando el puntero de pila: «ADD #Tam_RA_llamador, .SP» seguido de «MOVE .A, .SP».
4. Saltar al código de la función: «BR /funcion».
5. A la vuelta (etiqueta «ret»), restaurar el puntero de pila restándole Tam_RA_llamador y, si la función devuelve un valor, recogerlo.

El esquema de código generado para una llamada es el siguiente:

```

                                ; llamada funcion
MOVE #ret, #Tam_RA_llamador[.SP] ; guarda la direccion de retorno (EM)
MOVE #1[.SP], .R9                ; calcula el enlace estatico (PA)
MOVE .R9, #(Tam_RA_llamador+1)[.SP]
ADD #Tam_RA_llamador, .SP        ; activa el nuevo RA
MOVE .A, .SP
BR /funcion                      ; salta a la funcion
ret:
SUB .SP, #Tam_RA_llamador        ; restaura el SP al volver
MOVE .A, .SP

```

5.6 Retorno y recogida del valor devuelto

El retorno depende de la función y de si devuelve o no un valor:

- El retorno del programa principal (RETURN en «main») se traduce como HALT, deteniendo la ejecución.
- El retorno de un procedimiento que no devuelve valor (RETURN) se traduce como «BR [.SP]»: salta a la dirección de retorno almacenada en el campo EM (desplazamiento 0 del RA), devolviendo el control al llamador. Al no usarse Puntero de Control, el retorno es esta única instrucción.
- El retorno con valor (RETURN_ENT para enteros y RETURN_CAD para cadenas) copia primero el valor a devolver en la posición reservada para el valor devuelto que tanto el llamado como el llamador conocen por el tamaño del RA y a continuación ejecuta «BR [.SP]». Tras la vuelta, el llamador recoge ese valor y lo deja en el temporal indicado por el cuarteto de llamada.

Gracias a este funcionamiento de pila, las llamadas pueden anidarse y son combinadas con los bucles repeat-until, que permiten ejecutar correctamente programas con funciones recursivas o mutuamente recursivas y en cada invocación dispone de su propio registro de activación independiente en la cima de la pila, con sus parámetros y variables locales, sin interferir con las demás.

6.Casos de prueba

Una tabla resumen que tenga al menos 5 casos (En tu caso 6). Para cada caso, necesitamos una descripción, cuál es el programa fuente, y el resultado esperado. Una interpretación de cada uno. (Aquí no hace falta pegar el código, ya que eso se hará en el anexo)

Caso	Descripción	Programa	Resultado Esperado
1	Prueba general con IF, REPEAT-UNTIL, write y todos los tipos.	Prueba1.txt	Generar cuartetos ASIG, ASIG_CAD, GOTO_IG y GOTO

			para el IF y PRINT_CAD y PRINT_ENT para la salida.
2	Concatenación de strings con + dentro de REPEAT-UNTIL	Prueba2.txt	CONCAT en todos los + entre Strings, PRINT_CAD en writeln.
3	Concatenación de strings con + dentro de dos REPEAT-UNTIL anidados	Prueba3.txt	CONCAT en todos los + entre Strings y correcto funcionamiento de las etiquetas de los bucles en anidamiento.
4	IF dentro de un REPEAT-UNTIL	Prueba4.txt	Generar etiquetas del IF dentro del bucle REPEAT-UNTIL.
5	Dentro de un REPEAT-UNTIL, asignar valor a un String y concatenarlo en cada iteración	Prueba5.txt	Asignación debe generar ASIG_CAD y concatenar con CONCAT.
6	Llamada a una FUNCTION que recibe entero por valor y lo devuelve	Prueba6.txt	Generar ETIQ para la función, CALL_FUN para la llamada y PARAM para el parámetro que se manda.

7.Conclusiones

La realización de esta práctica nos ha permitido comprender el funcionamiento completo de un traductor de lenguajes, desde el análisis léxico, sintáctico y semántico hasta la generación de código objeto ensamblador ENS2025.

Hemos aprendido a integrar módulos independientes desarrollados en Java y Python, comunicados mediante ficheros con un formato de cuartetos estandarizado. Por un lado, diseñamos un Generador de Código Intermedio basado en el recorrido secuencial de tokens y el algoritmo de pila de precedencia para expresiones, implementando las construcciones específicas de nuestro grupo: la sentencia REPEAT-UNTIL, la concatenación de cadenas con el operador + y las funciones con paso de parámetros por valor. Por otro lado, el Generador de Código Objeto aplica plantillas fijas de instrucciones ENS2025, gestionando la memoria mediante zona estática y pila de registros de activación.

El proceso de depuración y verificación con casos de prueba progresivos nos ha permitido corregir problemas reales de desplazamientos, tipos y orden de emisión de etiquetas, consolidando los conocimientos adquiridos en la asignatura.

8.Anexo

Volcamos completo cada caso, con programa fuente, código intermedio, código objeto, salida de ejecución...

Prueba1.txt – Prueba General.

1.1 Programa fuente

```
PROGRAM pruebaCompleta;
VAR
    contador: integer;
    resultado: integer;
    mensaje: string;
    flag: boolean;
BEGIN
    contador := 0;
    resultado := 10;
    flag := true;
    mensaje := 'Inicio';

    { IF simple }
    IF flag THEN
        contador := contador + 1;

    { WRITE }
    write('Contador: ', contador);

    { REPEAT-UNTIL con = }
    REPEAT
        contador := contador + 1;
        resultado := resultado - 1;
    UNTIL contador = 5;

    { Sin WHILE (no tenéis >) }
    { Solo probamos con = }

    writeln('Final: ', contador);

    RETURN;
```

END;

1.2 Código intermedio

```
(ETIQ, {ET, "main"}, -, -)
(ASIG, {CTE_ENT, 0}, -, {VAR_LOCAL, 0})
(ASIG, {CTE_ENT, 10}, -, {VAR_LOCAL, 1})
(ASIG, {CTE_ENT, 1}, -, {VAR_LOCAL, 66})
(ASIG_CAD, {CTE_CAD, "Inicio"}, -, {VAR_LOCAL, 2})
(GOTO_IG, {VAR_LOCAL, 66}, {CTE_ENT, 1}, {ET, "L0"})
(GOTO, -, -, {ET, "L1"})
(ETIQ, {ET, "L0"}, -, -)
(ETIQ, {ET, "L1"}, -, -)
(SUMA, {VAR_LOCAL, 0}, {CTE_ENT, 1}, {VAR_TEMP, 0})
(ASIG, {VAR_TEMP, 0}, -, {VAR_LOCAL, 0})
(PRINT_CAD, -, -, {CTE_CAD, "Contador: "})
(PRINT_ENT, -, -, {VAR_LOCAL, 0})
(ETIQ, {ET, "L2"}, -, -)
(SUMA, {VAR_LOCAL, 0}, {CTE_ENT, 1}, {VAR_TEMP, 1})
(ASIG, {VAR_TEMP, 1}, -, {VAR_LOCAL, 0})
(RESTA, {VAR_LOCAL, 1}, {CTE_ENT, 1}, {VAR_TEMP, 2})
(ASIG, {VAR_TEMP, 2}, -, {VAR_LOCAL, 1})
(GOTO_IG, {VAR_LOCAL, 0}, {CTE_ENT, 5}, {ET, "L2"})
(PRINT_CAD, -, -, {CTE_CAD, "Final: "})
(PRINT_ENT, -, -, {VAR_LOCAL, 0})
(RETURN, -, -, -)
```

1.3 Código objeto

ORG 0

MOVE #inicio_estaticas, .IY

MOVE #inicio_pila, .SP

BR /main

main:

MOVE #0, #2[.SP]

MOVE #10, #3[.SP]

```

    MOVE #1, #68[.SP]

    MOVE #_str_0, .R0

    ADD #4, .SP

    MOVE .A, .R2
_scpy_1:
    CMP [.R0], #0

    BZ /_scpy_e_2

    MOVE [.R0], .R3

    MOVE .R3, [.R2]

    INC .R0

    INC .R2

    BR /_scpy_1
_scpy_e_2:
    MOVE #0, [.R2]

    CMP #68[.SP], #1

    BZ /L0

    BR /L1
L0:
L1:
    ADD #2[.SP], #1

    MOVE .A, #2[.SP]

    MOVE #2[.SP], #2[.SP]

    WRSTR /_str_1

    WRINT #2[.SP]
L2:
    ADD #2[.SP], #1

    MOVE .A, #3[.SP]

    MOVE #3[.SP], #2[.SP]

    SUB #3[.SP], #1

    MOVE .A, #4[.SP]

    MOVE #4[.SP], #3[.SP]

    CMP #2[.SP], #5

    BZ /L2

    WRSTR /_str_2

    WRINT #2[.SP]

    HALT

```

```

; Segmento de datos
_str_0: DATA "Inicio"
_str_1: DATA "Contador: "
_str_2: DATA "Final: "
inicio_estaticas: RES 1
inicio_pila:
END

```

Prueba2.txt - Prueba concatenación

2.1 Programa fuente

```

PROGRAM pruebaConcatenacion;
VAR
    s1: string;
    s2: string;
    res: string;
    i: integer;
BEGIN
    s1 := 'Hola';
    s2 := ' Mundo';
    res := s1 + s2;
    i := 0;
    REPEAT
        res := res + '!';
        i := i + 1;
    UNTIL i = 3;
    writeln(res);
    RETURN;
END;

```

2.2 Código intermedio

```

(ETIQ, {ET, "main"}, -, -)
(ASIG_CAD, {CTE_CAD, "Hola"}, -, {VAR_LOCAL, 0})
(ASIG_CAD, {CTE_CAD, " Mundo"}, -, {VAR_LOCAL, 64})
(CONCAT, {VAR_LOCAL, 0}, {VAR_LOCAL, 64}, {VAR_TEMP, 0})

```

```

(ASIG_CAD, {VAR_TEMP, 0}, -, {VAR_LOCAL, 128})
(ASIG, {CTE_ENT, 0}, -, {VAR_LOCAL, 192})
(ETIQ, {ET, "L0"}, -, -)
(CONCAT, {VAR_LOCAL, 128}, {CTE_CAD, "!"}, {VAR_TEMP, 1})
(ASIG_CAD, {VAR_TEMP, 1}, -, {VAR_LOCAL, 128})
(SUMA, {VAR_LOCAL, 192}, {CTE_ENT, 1}, {VAR_TEMP, 2})
(ASIG, {VAR_TEMP, 2}, -, {VAR_LOCAL, 192})
(GOTO_IG, {VAR_LOCAL, 192}, {CTE_ENT, 3}, {ET, "L0"})
(PRINT_CAD, -, -, {VAR_LOCAL, 128})
(RETURN, -, -, -)

```

2.3 Código objeto

```

ORG 0

    MOVE #inicio_estaticas, .IY
    MOVE #inicio_pila, .SP
    BR /main

```

main:

```

    MOVE #_str_0, .R0
    ADD #2, .SP
    MOVE .A, .R2

```

_scopy_1:

```

    CMP [.R0], #0
    BZ /_scopy_e_2
    MOVE [.R0], .R3
    MOVE .R3, [.R2]
    INC .R0
    INC .R2
    BR /_scopy_1

```

_scopy_e_2:

```

    MOVE #0, [.R2]
    MOVE #_str_1, .R0
    ADD #66, .SP
    MOVE .A, .R2

```

_scopy_3:

```

    CMP [.R0], #0
    BZ /_scopy_e_4
    MOVE [.R0], .R3
    MOVE .R3, [.R2]
    INC .R0
    INC .R2
    BR /_scopy_3
_scopy_e_4:
    MOVE #0, [.R2]
    ADD #2, .SP
    MOVE .A, .R0
    ADD #66, .SP
    MOVE .A, .R1
    ADD #2, .SP
    MOVE .A, .R2
_cat1_5:
    CMP [.R0], #0
    BZ /_cat1e_6
    MOVE [.R0], .R3
    MOVE .R3, [.R2]
    INC .R0
    INC .R2
    BR /_cat1_5
_cat1e_6:
    CMP [.R1], #0
    BZ /_cat2e_7
    MOVE [.R1], .R3
    MOVE .R3, [.R2]
    INC .R1
    INC .R2
    BR /_cat1e_6
_cat2e_7:
    MOVE #0, [.R2]
    ADD #2, .SP
    MOVE .A, .R0
    ADD #130, .SP

```

```

        MOVE .A, .R2
_scpy_8:
        CMP [.R0], #0
        BZ /_scopy_e_9
        MOVE [.R0], .R3
        MOVE .R3, [.R2]
        INC .R0
        INC .R2
        BR /_scopy_8
_scpy_e_9:
        MOVE #0, [.R2]
        MOVE #0, #194[.SP]
L0:
        ADD #130, .SP
        MOVE .A, .R0
        MOVE #_str_2, .R1
        ADD #3, .SP
        MOVE .A, .R2
_cat1_10:
        CMP [.R0], #0
        BZ /_cat1e_11
        MOVE [.R0], .R3
        MOVE .R3, [.R2]
        INC .R0
        INC .R2
        BR /_cat1_10
_cat1e_11:
        CMP [.R1], #0
        BZ /_cat2e_12
        MOVE [.R1], .R3
        MOVE .R3, [.R2]
        INC .R1
        INC .R2
        BR /_cat1e_11
_cat2e_12:
        MOVE #0, [.R2]

```

```

    ADD #3, .SP

    MOVE .A, .R0

    ADD #130, .SP

    MOVE .A, .R2
_scpy_13:
    CMP [.R0], #0

    BZ /_scpy_e_14

    MOVE [.R0], .R3

    MOVE .R3, [.R2]

    INC .R0

    INC .R2

    BR /_scpy_13
_scpy_e_14:
    MOVE #0, [.R2]

    ADD #194[.SP], #1

    MOVE .A, #4[.SP]

    MOVE #4[.SP], #194[.SP]

    CMP #194[.SP], #3

    BZ /L0

    WRSTR #130[.SP]

    HALT

; Segmento de datos
_str_0: DATA "Hola"
_str_1: DATA " Mundo"
_str_2: DATA "!"
inicio_estaticas: RES 1
inicio_pila:
END

```

Prueba3.txt - pruebaConcatRepeat

3.1 Programa fuente

```

PROGRAM pruebaConcatRepeat;

VAR

    prefijo: string;

```

```

sufijo: string;
resultado: string;
separador: string;
i: integer;
j: integer;
BEGIN
    prefijo := 'Vuelta';
    sufijo := 'fin';
    separador := '-';
    i := 1;
    REPEAT
        resultado := prefijo + separador;
        j := 0;
        REPEAT
            resultado := resultado + sufijo;
            j := j + 1;
        UNTIL j = i;
        writeln(resultado);
        i := i + 1;
    UNTIL i = 4;
    RETURN;
END;

```

3.2 Código intermedio

```

(ETIQ, {ET, "main"}, -, -)
(ASIG_CAD, {CTE_CAD, "Vuelta"}, -, {VAR_LOCAL, 0})
(ASIG_CAD, {CTE_CAD, "fin"}, -, {VAR_LOCAL, 64})
(ASIG_CAD, {CTE_CAD, "-"}, -, {VAR_LOCAL, 192})
(ASIG, {CTE_ENT, 1}, -, {VAR_LOCAL, 256})
(ETIQ, {ET, "L0"}, -, -)
(CONCAT, {VAR_LOCAL, 0}, {VAR_LOCAL, 192}, {VAR_TEMP, 0})
(ASIG_CAD, {VAR_TEMP, 0}, -, {VAR_LOCAL, 128})
(ASIG, {CTE_ENT, 0}, -, {VAR_LOCAL, 257})
(ETIQ, {ET, "L1"}, -, -)
(CONCAT, {VAR_LOCAL, 128}, {VAR_LOCAL, 64}, {VAR_TEMP, 1})

```

```

(ASIG_CAD, {VAR_TEMP, 1}, -, {VAR_LOCAL, 128})
(SUMA, {VAR_LOCAL, 257}, {CTE_ENT, 1}, {VAR_TEMP, 2})
(ASIG, {VAR_TEMP, 2}, -, {VAR_LOCAL, 257})
(GOTO_IG, {VAR_LOCAL, 257}, {VAR_LOCAL, 256}, {ET, "L1"})
(PRINT_CAD, -, -, {VAR_LOCAL, 128})
(SUMA, {VAR_LOCAL, 256}, {CTE_ENT, 1}, {VAR_TEMP, 3})
(ASIG, {VAR_TEMP, 3}, -, {VAR_LOCAL, 256})
(GOTO_IG, {VAR_LOCAL, 256}, {CTE_ENT, 4}, {ET, "L0"})
(RETURN, -, -, -)

```

3.3 Código fuente

ORG 0

```

    MOVE #inicio_estaticas, .IY
    MOVE #inicio_pila, .SP
    BR /main

```

main:

```

    MOVE #_str_0, .R0
    ADD #2, .SP
    MOVE .A, .R2

```

_scopy_1:

```

    CMP [.R0], #0
    BZ /_scopy_e_2
    MOVE [.R0], .R3
    MOVE .R3, [.R2]
    INC .R0
    INC .R2
    BR /_scopy_1

```

_scopy_e_2:

```

    MOVE #0, [.R2]
    MOVE #_str_1, .R0
    ADD #66, .SP
    MOVE .A, .R2

```

_scopy_3:

```

    CMP [.R0], #0

```

```

    BZ /_scopy_e_4
    MOVE [.R0], .R3
    MOVE .R3, [.R2]
    INC .R0
    INC .R2
    BR /_scopy_3
_scopy_e_4:
    MOVE #0, [.R2]
    MOVE #_str_2, .R0
    ADD #194, .SP
    MOVE .A, .R2
_scopy_5:
    CMP [.R0], #0
    BZ /_scopy_e_6
    MOVE [.R0], .R3
    MOVE .R3, [.R2]
    INC .R0
    INC .R2
    BR /_scopy_5
_scopy_e_6:
    MOVE #0, [.R2]
    MOVE #1, #258[.SP]
L0:
    ADD #2, .SP
    MOVE .A, .R0
    ADD #194, .SP
    MOVE .A, .R1
    ADD #2, .SP
    MOVE .A, .R2
_cat1_7:
    CMP [.R0], #0
    BZ /_cat1e_8
    MOVE [.R0], .R3
    MOVE .R3, [.R2]
    INC .R0
    INC .R2

```

```

        BR /_cat1_7
_cat1e_8:
        CMP [.R1], #0
        BZ /_cat2e_9
        MOVE [.R1], .R3
        MOVE .R3, [.R2]
        INC .R1
        INC .R2
        BR /_cat1e_8
_cat2e_9:
        MOVE #0, [.R2]
        ADD #2, .SP
        MOVE .A, .R0
        ADD #130, .SP
        MOVE .A, .R2
_scpy_10:
        CMP [.R0], #0
        BZ /_scopy_e_11
        MOVE [.R0], .R3
        MOVE .R3, [.R2]
        INC .R0
        INC .R2
        BR /_scopy_10
_scpy_e_11:
        MOVE #0, [.R2]
        MOVE #0, #259[.SP]
L1:
        ADD #130, .SP
        MOVE .A, .R0
        ADD #66, .SP
        MOVE .A, .R1
        ADD #3, .SP
        MOVE .A, .R2
_cat1_12:
        CMP [.R0], #0
        BZ /_cat1e_13

```

```

    MOVE [.R0], .R3
    MOVE .R3, [.R2]
    INC .R0
    INC .R2
    BR /_cat1_12
_cat1e_13:
    CMP [.R1], #0
    BZ /_cat2e_14
    MOVE [.R1], .R3
    MOVE .R3, [.R2]
    INC .R1
    INC .R2
    BR /_cat1e_13
_cat2e_14:
    MOVE #0, [.R2]
    ADD #3, .SP
    MOVE .A, .R0
    ADD #130, .SP
    MOVE .A, .R2
_scpy_15:
    CMP [.R0], #0
    BZ /_scopy_e_16
    MOVE [.R0], .R3
    MOVE .R3, [.R2]
    INC .R0
    INC .R2
    BR /_scopy_15
_scpy_e_16:
    MOVE #0, [.R2]
    ADD #259[.SP], #1
    MOVE .A, #4[.SP]
    MOVE #4[.SP], #259[.SP]
    CMP #259[.SP], #258[.SP]
    BZ /L1
    WRSTR #130[.SP]
    ADD #258[.SP], #1

```

```

    MOVE .A, #5[.SP]

    MOVE #5[.SP], #258[.SP]

    CMP #258[.SP], #4

    BZ /L0

    HALT

; Segmento de datos
_str_0: DATA "Vuelta"
_str_1: DATA "fin"
_str_2: DATA "-"
inicio_estaticas: RES 1
inicio_pila:
END

```

Prueba4.txt - PruebaCondiciones

4.1 Programa fuente

```

PROGRAM pruebaCondiciones;

VAR

    a: integer;

    b: integer;

    resultado: integer;

BEGIN

    a := 10;

    b := 10;

    resultado := 0;

    REPEAT

        IF a = b THEN

            resultado := resultado + 1;

            a := a - 1;

            b := b - 2;

        UNTIL a = 0;

    RETURN;

END;

```

4.2 Código intermedio

```

(ETIQ, {ET, "main"}, -, -)
(ASIG, {CTE_ENT, 10}, -, {VAR_LOCAL, 0})
(ASIG, {CTE_ENT, 10}, -, {VAR_LOCAL, 1})
(ASIG, {CTE_ENT, 0}, -, {VAR_LOCAL, 2})
(ETIQ, {ET, "L0"}, -, -)
(GOTO_IG, {VAR_LOCAL, 0}, {VAR_LOCAL, 1}, {ET, "L1"})
(GOTO, -, -, {ET, "L2"})
(ETIQ, {ET, "L1"}, -, -)
(ETIQ, {ET, "L2"}, -, -)
(SUMA, {VAR_LOCAL, 2}, {CTE_ENT, 1}, {VAR_TEMP, 0})
(ASIG, {VAR_TEMP, 0}, -, {VAR_LOCAL, 2})
(RESTA, {VAR_LOCAL, 0}, {CTE_ENT, 1}, {VAR_TEMP, 1})
(ASIG, {VAR_TEMP, 1}, -, {VAR_LOCAL, 0})
(RESTA, {VAR_LOCAL, 1}, {CTE_ENT, 2}, {VAR_TEMP, 2})
(ASIG, {VAR_TEMP, 2}, -, {VAR_LOCAL, 1})
(GOTO_IG, {VAR_LOCAL, 0}, {CTE_ENT, 0}, {ET, "L0"})
(RETURN, -, -, -)

```

4.3 Código objeto

ORG 0

```

    MOVE #inicio_estaticas, .IY
    MOVE #inicio_pila, .SP
    BR /main

```

main:

```

    MOVE #10, #2[.SP]
    MOVE #10, #3[.SP]
    MOVE #0, #4[.SP]

```

L0:

```

    CMP #2[.SP], #3[.SP]
    BZ /L1
    BR /L2

```

L1:

L2:

```

    ADD #4[.SP], #1

```

```

    MOVE .A, #2[.SP]
    MOVE #2[.SP], #4[.SP]
    SUB #2[.SP], #1
    MOVE .A, #3[.SP]
    MOVE #3[.SP], #2[.SP]
    SUB #3[.SP], #2
    MOVE .A, #4[.SP]
    MOVE #4[.SP], #3[.SP]
    CMP #2[.SP], #0
    BZ /L0
    HALT

; Segmento de datos
inicio_estaticas: RES 1
inicio_pila:
END

```

Prueba5.txt - pruebaStringRepeat

5.1 Programa fuente

```

PROGRAM pruebaStringRepeat;
VAR
    s: string;
    copia: string;
    i: integer;
BEGIN
    s := 'AB';
    i := 0;
    REPEAT
        copia := s;
        s := copia + copia;
        i := i + 1;
    UNTIL i = 3;
    writeln(s);
    RETURN;
END;

```

5.2 Código intermedio

```
(ETIQ, {ET, "main"}, -, -)
(ASIG_CAD, {CTE_CAD, "AB"}, -, {VAR_LOCAL, 0})
(ASIG, {CTE_ENT, 0}, -, {VAR_LOCAL, 128})
(ETIQ, {ET, "L0"}, -, -)
(ASIG_CAD, {VAR_LOCAL, 0}, -, {VAR_LOCAL, 64})
(CONCAT, {VAR_LOCAL, 64}, {VAR_LOCAL, 64}, {VAR_TEMP, 0})
(ASIG_CAD, {VAR_TEMP, 0}, -, {VAR_LOCAL, 0})
(SUMA, {VAR_LOCAL, 128}, {CTE_ENT, 1}, {VAR_TEMP, 1})
(ASIG, {VAR_TEMP, 1}, -, {VAR_LOCAL, 128})
(GOTO_IG, {VAR_LOCAL, 128}, {CTE_ENT, 3}, {ET, "L0"})
(PRINT_CAD, -, -, {VAR_LOCAL, 0})
(RETURN, -, -, -)
```

5.3 Código objeto

ORG 0

```
MOVE #inicio_estaticas, .IY
MOVE #inicio_pila, .SP
BR /main
```

main:

```
MOVE #_str_0, .R0
ADD #2, .SP
MOVE .A, .R2
```

_scopy_1:

```
CMP [.R0], #0
BZ /_scopy_e_2
MOVE [.R0], .R3
MOVE .R3, [.R2]
INC .R0
INC .R2
BR /_scopy_1
```

_scopy_e_2:

```
MOVE #0, [.R2]
```

```

        MOVE #0, #130[.SP]
L0:
        ADD #2, .SP
        MOVE .A, .R0
        ADD #66, .SP
        MOVE .A, .R2
_scpy_3:
        CMP [.R0], #0
        BZ /_scopy_e_4
        MOVE [.R0], .R3
        MOVE .R3, [.R2]
        INC .R0
        INC .R2
        BR /_scopy_3
_scpy_e_4:
        MOVE #0, [.R2]
        ADD #66, .SP
        MOVE .A, .R0
        ADD #66, .SP
        MOVE .A, .R1
        ADD #2, .SP
        MOVE .A, .R2
_cat1_5:
        CMP [.R0], #0
        BZ /_cat1e_6
        MOVE [.R0], .R3
        MOVE .R3, [.R2]
        INC .R0
        INC .R2
        BR /_cat1_5
_cat1e_6:
        CMP [.R1], #0
        BZ /_cat2e_7
        MOVE [.R1], .R3
        MOVE .R3, [.R2]
        INC .R1

```

```

        INC .R2

        BR /_cat1e_6
_cat2e_7:
        MOVE #0, [.R2]

        ADD #2, .SP

        MOVE .A, .R0

        ADD #2, .SP

        MOVE .A, .R2

_scpy_8:
        CMP [.R0], #0

        BZ /_scopy_e_9

        MOVE [.R0], .R3

        MOVE .R3, [.R2]

        INC .R0

        INC .R2

        BR /_scopy_8

_scpy_e_9:
        MOVE #0, [.R2]

        ADD #130[.SP], #1

        MOVE .A, #3[.SP]

        MOVE #3[.SP], #130[.SP]

        CMP #130[.SP], #3

        BZ /L0

        WRSTR #2[.SP]

        HALT

; Segmento de datos
_str_0: DATA "AB"
inicio_estaticas: RES 1
inicio_pila:
END

```

Prueba6.txt - pruebaFuncion

6.1 Programa fuente

```

FUNCTION doble(n: integer): integer;

```

```

BEGIN

    RETURN n;

END;

PROGRAM pruebaFuncion;

VAR

    x: integer;

BEGIN

    x := doble(5);

    RETURN;

END;

```

6.2 Código intermedio

```

(ETIQ, {ET, "Etiqueta0"}, -, -)
(RETURN_ENT, -, -, {VAR_LOCAL, 0})
(ETIQ, {ET, "main"}, -, -)
(PARAM, {CTE_ENT, 5}, -, -)
(CALL_FUN, {ET, "Etiqueta0"}, -, {VAR_TEMP, 0})
(ASIG, {VAR_TEMP, 0}, -, {VAR_LOCAL, 0})
(RETURN, -, -, -)

```

6.3 Código fuente

```

ORG 0

    MOVE #inicio_estaticas, .IY

    MOVE #inicio_pila, .SP

    BR /main

```

```

Etiqueta0:

    SUB #3, #1

    ADD .A, .SP

    MOVE #2[.SP], [.A]

    BR [.SP]

```

```

main:

    ADD #5, .SP

    MOVE #5, [.A]

; llamada Etiqueta0

```

```

    MOVE #_ret_1, #3[.SP]

    MOVE .SP, .R9

    MOVE .R9, #4[.SP]

    ADD #3, .SP

    MOVE .A, .SP

    BR /Etiqueta0

_ret_1:

    SUB #3, #1

    ADD .A, .SP

    MOVE [.A], .R9

    SUB .SP, #3

    MOVE .A, .SP

    MOVE .R9, #2[.SP]

    MOVE #2[.SP], #2[.SP]

    HALT

; Segmento de datos

inicio_estaticas: RES 1

inicio_pila:

END

```